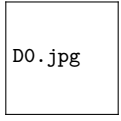
Lecture 02: From Files to Databases

Origins of Database Management Systems

Z2004: Database Management Systems

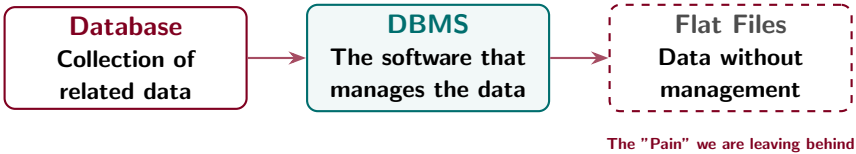


IIT Madras Zanzibar

Tuesday, March 3, 2026

Lecture 01 Recap: The Core Anchors

Before we dive deeper into RDBMS, let's confirm these three fundamental definitions.



- **Data vs. Information:** We discussed how a DBMS turns raw bits into meaningful, searchable information.
- **The Redundancy Trap:** We saw how duplicate data in flat files leads to *Inconsistency*—where the same student has two different names in different files.

Learning Outcomes: What You Will Master

By the end of this lecture, you will be able to:

- **Critique Flat Files:** Identify the 5 critical failures of file-based systems, including redundancy and security gaps.
- **Define the Relational Model:** Explain the significance of E.F. Codd's 1970 paper and how it distinguishes an RDBMS from a basic DBMS.
- **Design Better Tables:** Convert a redundant "Flat File" dataset into a structured, relational design using shared keys.
- **Implement Access Control:** Understand how `GRANT` and `REVOKE` commands solve the "No Security" failure in multi-user environments.

Today's Journey

From the chaos of 1960s flat files to the birth of the modern DBMS.

- **1. The World Before DBMS**
  - Data, Information, Knowledge
  - The 1960s university scenario
  - Why flat files failed
- **2. Five Failures**
  - Redundancy, Inconsistency, Isolation
  - No Atomicity, No Security
  - Each failure drove one DBMS innovation
- **3. DBMS vs RDBMS**
  - Files vs Tables with Foreign Keys
  - The Relational Model (Codd, 1970)
  - The 60-year evolution timeline
- **4. Activity + Exit Quiz**
  - Think-Pair-Share (7 min)
  - 3-question exit quiz

Core question today: Why did the world **have** *to invent the DBMS?*

SECTION 1

The World Before DBMS

Data, Information, and the Flat-File Era

What is Data?

Data is the raw material.

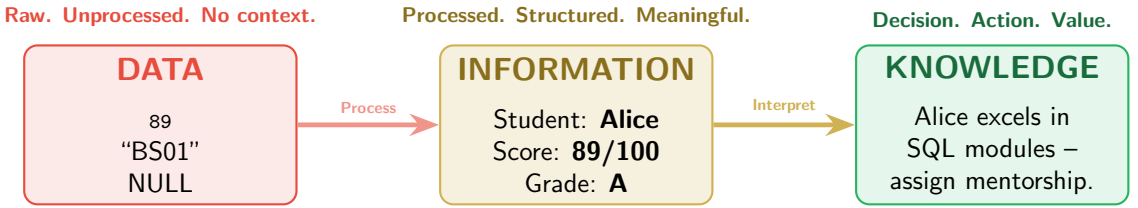
Raw facts or figures that have no meaning on their own.

Raw Data
"39"
"BS24"
"101"

+ Context →

Meaningful Information
Patient Temp: **39°C**
Student Roll: **BS24**
Room Number: **101**

Three Levels: Data, Information, Knowledge



The DBMS lives at the boundary: it converts Data into Information.

What is a Database?

A collection of related data built for **speed**, **search**, and **scale**.

Structured
Organized into strict rows and columns, not messy files.

Accessible
Instantly queried, filtered, and updated by users.

Secure
Protected from crashes and unauthorized eyes.

The Anatomy of a Table

1. Table (Relation)
The entire grid holding entity data.

2. Record (Row/Tuple)
One complete horizontal entry representing one entity.

3. Field (Column/Attribute)
A vertical category of data sharing the same type.

→ Highlighting a Record (Row)

Roll No.	Name	Age
BS01	Peter	19
BS24	**Anita**	**20**
BS33	John	21

↓ Highlighting a Field (Column)

Roll No.	Name	Age
BS01	Peter	**19**
BS24	Anita	**20**
BS33	John	**21**

The Power of Relational Data

How do we find the youngest student's hostel? We link tables!

Students

Roll	Name	Age
BS1	John	20
BS2	Alice	22
BS3	Bob	**19**

Residency

Roll	Hostel
BS1	Block A
BS2	Block B
BS3	**Block A**

The 1960s University: A Real Scenario

IIT Madras, circa 1960

No databases. Data lives in separate OS files.

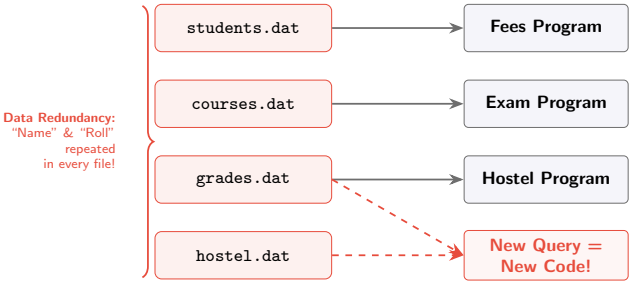
`students.dat`	Name, Roll, Age
`courses.dat`	CourseID, Credits
`grades.dat`	Roll, Score, Grade
`hostel.dat`	Roll, Room, Block

The Registrar asks:

> *"Which hostel block does the highest-scoring student live in?"*

To answer this, you must:

1. Read `grades.dat` & `hostel.dat`
2. Write a custom COBOL program.
3. Wait 3 days for the programmer.

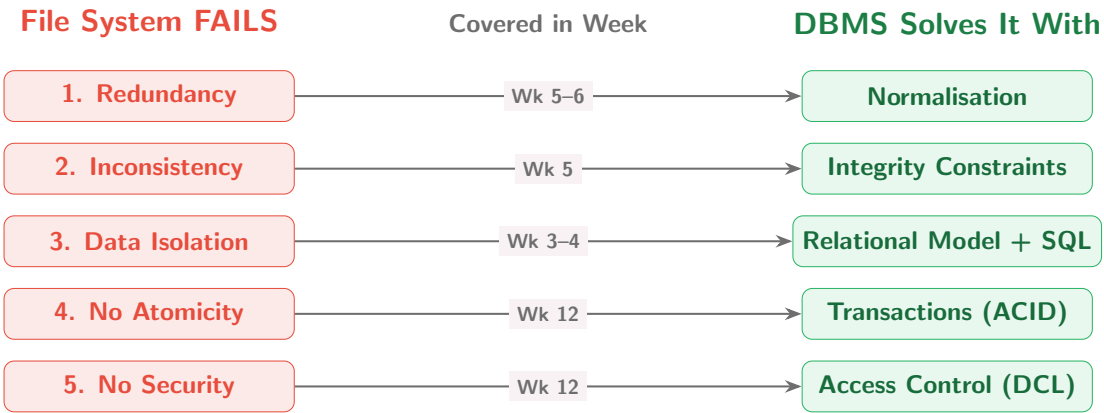


SECTION 2

Five Failures

Each failure drove exactly one DBMS innovation.

Five Failures, Five Innovations



This course is the story of how each of these five problems was solved.

Failure 1: Redundancy

The Problem

"Alice Mwanga" is copied in 4 files:

`payroll.dat`	Alice, 28, CS
`library.dat`	Alice, 28, CS
`hostel.dat`	Alice, 28, CS
`exam.dat`	Alice, 28, CS

Consequences:

- Higher storage cost
- Update one – miss others
- Inconsistency guaranteed

DBMS Fix:

One `Student` table (one row per student).
Other tables reference it via a **Primary Key.**

Failure 1: Redundancy

The Problem

"Alice Mwanga" is copied in 4 files:

`payroll.dat`	Alice, 28, CS
`library.dat`	Alice, 28, CS
`hostel.dat`	Alice, 28, CS
`exam.dat`	Alice, 28, CS

Consequences:

- Higher storage cost
- Update one – miss others
- Inconsistency guaranteed

DBMS Fix:

One `Student` table (one row per student).
Other tables reference it via a **Primary Key.**

File Era

`payroll.dat: Alice, 28, CS`

`library.dat: Alice, 28, CS`

`hostel.dat: Alice, 28, CS`

`exam.dat: Alice, 28, CS`

× 4 copies – disaster waiting

Failure 1: Redundancy

The Problem

"Alice Mwanga" is copied in 4 files:

`payroll.dat`	Alice, 28, CS
`library.dat`	Alice, 28, CS
`hostel.dat`	Alice, 28, CS
`exam.dat`	Alice, 28, CS

Consequences:

- Higher storage cost
- Update one – miss others
- Inconsistency guaranteed

DBMS Fix:

One `Student` table (one row per student).
Other tables reference it via a **Primary Key.**

File Era

`payroll.dat: Alice, 28, CS`

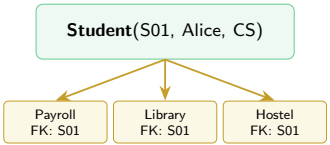
`library.dat: Alice, 28, CS`

`hostel.dat: Alice, 28, CS`

`exam.dat: Alice, 28, CS`

× 4 copies – disaster waiting

DBMS Era



✓ 1 copy. Always consistent.

Failures 2 & 3: Inconsistency and Isolation

× Failure 2: Inconsistency

Alice changes address to **Nungwi.**

Payroll updates it. Library forgets.

`payroll.dat:` Nungwi
`library.dat:` Paje ← **WRONG**

DBMS Fix: One source of truth.
Update once – all references see it.

Failures 2 & 3: Inconsistency and Isolation

× Failure 2: Inconsistency

Alice changes address to **Nungwi.**

Payroll updates it. Library forgets.

`payroll.dat:` Nungwi
`library.dat:` Paje ← **WRONG**

DBMS Fix: One source of truth.
Update once – all references see it.

× Failure 3: Data Isolation

"List students in Block A who scored above 80."

Needs `grades.dat` + `hostel.dat`
Different formats. No shared key.
→ Write a new COBOL program.
→ 3 days of programmer time.

DBMS Fix: One SQL JOIN query.
`SELECT...JOIN hostel ON Roll`

Failures 2 & 3: Inconsistency and Isolation

× Failure 2: Inconsistency

Alice changes address to **Nungwi.**

Payroll updates it. Library forgets.

`payroll.dat:` Nungwi
`library.dat:` Paje ← **WRONG**

DBMS Fix: One source of truth.
Update once – all references see it.

× Failure 3: Data Isolation

"List students in Block A who scored above 80."

Needs `grades.dat` + `hostel.dat`
Different formats. No shared key.
→ Write a new COBOL program.
→ 3 days of programmer time.

DBMS Fix: One SQL JOIN query.
`SELECT...JOIN hostel ON Roll`

Root cause of both: no shared, unified data model. The relational model solved this in 1970.

Failure 4: No Atomicity – The Crash Scenario

The M-Pesa Transfer

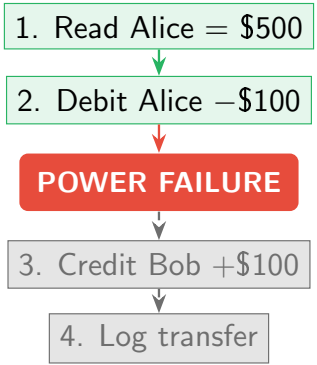
From Lecture 01.

5 steps. Must complete as ONE unit.

1. Read Alice's balance: \$500
2. Debit Alice: −\$100

✕ **POWER FAILS HERE.**

3. Credit Bob: *never happens.*
4. Log: *never written.*



Failure 4: No Atomicity – The Crash Scenario

The M-Pesa Transfer

From Lecture 01.

5 steps. Must complete as ONE unit.

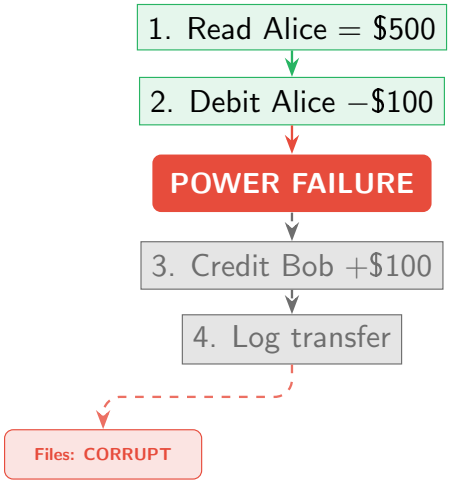
1. Read Alice's balance: \$500
2. Debit Alice: −\$100

✕ **POWER FAILS HERE.**

3. Credit Bob: *never happens.*
4. Log: *never written.*

File System (Fails):

Alice lost \$100. Bob got nothing.
Data is permanently corrupted.



Failure 4: No Atomicity – The Crash Scenario

The M-Pesa Transfer

From Lecture 01.

5 steps. Must complete as ONE unit.

1. Read Alice's balance: \$500
2. Debit Alice: −\$100

✕ **POWER FAILS HERE.**

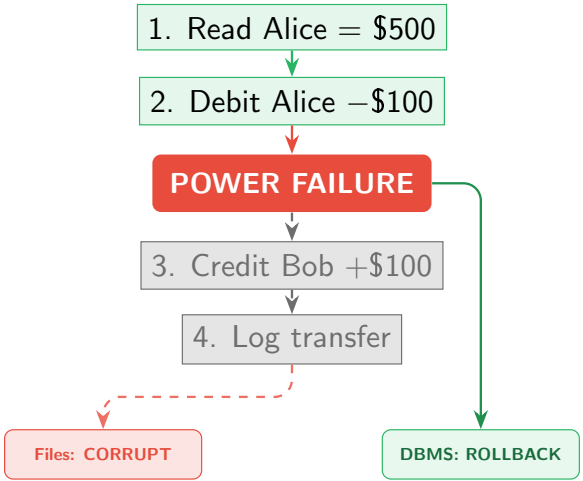
3. Credit Bob: *never happens.*
4. Log: *never written.*

File System (Fails):

Alice lost \$100. Bob got nothing.
Data is permanently corrupted.

DBMS (Succeeds):

Transaction **rolled back** entirely.
Alice's balance is restored to \$500.



Failure 5: No Security

The Problem

Any program that opens a file reads everything.
A payroll clerk could read `medical.dat`.
No logs. No audit trail. No roles.

Failure 5: No Security

The Problem

Any program that opens a file reads everything.
A payroll clerk could read `medical.dat`.
No logs. No audit trail. No roles.

The DBMS Solution

GRANT and **REVOKE** specific permissions.
Each user sees only their authorised view.

Failure 5: No Security

The Problem

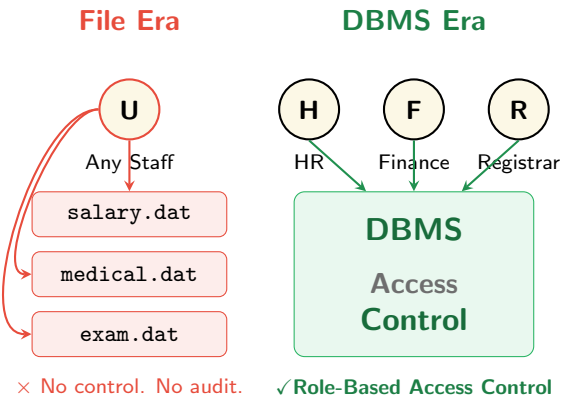
Any program that opens a file reads everything.
A payroll clerk could read `medical.dat`.
No logs. No audit trail. No roles.

The DBMS Solution

GRANT and **REVOKE** specific permissions.
Each user sees only their authorised view.

```
-- HR sees staff records only
GRANT SELECT ON Staff TO hr_role;

-- Registrar sees grades only
GRANT SELECT ON Grades TO registrar;
```



Visual Representation of Data Access

SECTION 3

DBMS vs RDBMS

Files vs Tables – and Why Relationships Changed Everything

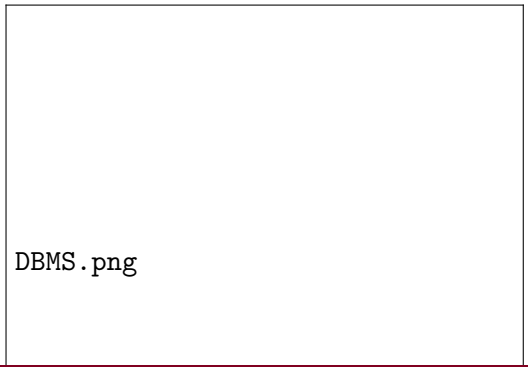
What is a DBMS?

Definition

Software that interacts with users, applications, and the database itself to capture, store, retrieve, and analyze data.

Core Functions:

- Efficient Storage & Retrieval
- Data Integrity & Security
- Concurrent Multi-User Access

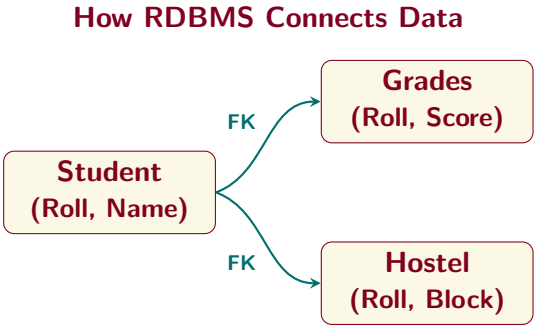


DBMS vs RDBMS: The Core Differences

Property	DBMS	RDBMS
Data Model	Files, hierarchical, network	Tables with rows and columns
Relationships	Not enforced	Enforced via **Foreign Keys**
Scale	Small, single-user	Large, multi-user, concurrent
Query Language	Custom programs (COBOL)	**SQL** (standardised 1974)
Redundancy	High, uncontrolled	Controlled via **Normalisation**
Examples	Early IBM IMS, XML files	PostgreSQL, MySQL, MS SQL Server

The Relational Database Management System (RDBMS) is the modern standard.

The Relational Shift & Our Course Focus



One unified record.
Many connected relationships.

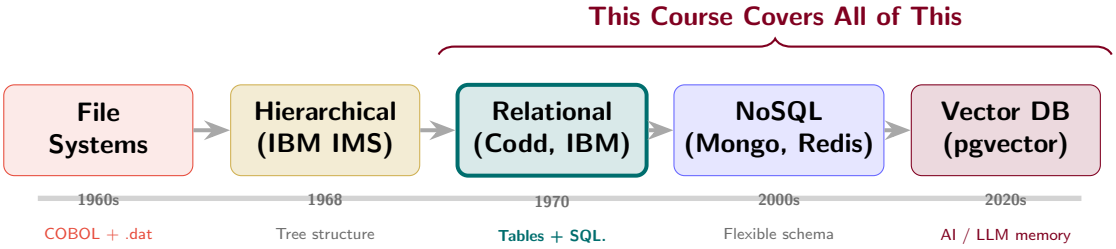
In this course:

We study the **Relational Model** introduced by E.F. Codd (IBM, 1970).

Our Toolkit:

- **Microsoft SQL Server 2022**
  (Phase 1 & 2)
- **PostgreSQL**
  (Phase 3)

60 Years in One Slide: The Database Family Tree



Read Silberschatz, Korth & Sudarshan Ch. 1 – it covers this full arc.

RDBMS in the Real World

$ Banking & Finance

Core Feature: Data Integrity

Prevents money from vanishing during simultaneous transfers.

RDBMS in the Real World

$ Banking & Finance

Core Feature: Data Integrity

Prevents money from vanishing during simultaneous transfers.

U University Systems

Core Feature: Relationships

Links thousands of students, grades, and hostels without redundancy.

RDBMS in the Real World

$ Banking & Finance

Core Feature: Data Integrity

Prevents money from vanishing during simultaneous transfers.

U University Systems

Core Feature: Relationships

Links thousands of students, grades, and hostels without redundancy.

E E-Commerce

Core Feature: Concurrency

Ensures two people don't buy the exact same "last item in stock".

RDBMS in the Real World

$ Banking & Finance

Core Feature: Data Integrity

Prevents money from vanishing during simultaneous transfers.

U University Systems

Core Feature: Relationships

Links thousands of students, grades, and hostels without redundancy.

E E-Commerce

Core Feature: Concurrency

Ensures two people don't buy the exact same "last item in stock".

+ Healthcare

Core Feature: Security & Access

Strict role-based views. Doctors see charts, billing sees invoices.

Banking & Finance

Core Feature: Data Integrity

Prevents money from vanishing during simultaneous transfers.

RDBMS in the Real World

Banking & Finance

Core Feature: Data Integrity

Prevents money from vanishing during simultaneous transfers.

University Systems

Core Feature: Relationships

Links thousands of students, grades, and hostels without redundancy.

RDBMS in the Real World

Banking & Finance

Core Feature: Data Integrity

Prevents money from vanishing during simultaneous transfers.

University Systems

Core Feature: Relationships

Links thousands of students, grades, and hostels without redundancy.

E-Commerce

Core Feature: Concurrency

Ensures two people don't buy the exact same "last item in stock".

RDBMS in the Real World

Banking & Finance

Core Feature: Data Integrity

Prevents money from vanishing during simultaneous transfers.

University Systems

Core Feature: Relationships

Links thousands of students, grades, and hostels without redundancy.

E-Commerce

Core Feature: Concurrency

Ensures two people don't buy the exact same "last item in stock".

Healthcare

Core Feature: Security & Access

Strict role-based views. Doctors see charts, billing sees invoices.

In-Class Activity: Think – Pair – Share

4 Minutes — Discuss with your neighbour

Scenario: IIT Madras Zanzibar Student Portal

The current system uses three flat files: `students.dat`, `grades.dat`, and `fees.dat`.
The student's **name** and **roll number** are stored in all three files.

1. **Which of the 5 failures is this system guilty of?**
2. **Sketch a better table design on paper.**
   How many tables? What columns? What is the shared key?
3. **A student changes their name. Compare the effort:**
   Flat file approach vs. DBMS approach.

Summary: The Big Picture

The Pain of Flat Files

- ✗ Massive Data Redundancy
- ✗ Inconsistent Updates
- ✗ Zero Concurrent Control
- ✗ Hard-coded Queries
- ✗ Poor Security & Access

The RDBMS Promise

- ✓ Normalized, clean data
- ✓ ACID Properties (Integrity)
- ✓ Safe Multi-user Access
- ✓ Standardized SQL
- ✓ Role-based Security

Takeaway: Databases don't just "store" data; they **protect, organize, and serve** it efficiently at scale.

Exit Quiz: 3 Questions — 3 Minutes

⧗ Quick check before we wrap up Lecture 02.

Q1. Student data appears in multiple files with the name duplicated. Failure?

(a) Atomicity
(b) Redundancy
(c) Isolation
(d) Security

Q2. Which 1970 model became the foundation of SQL?

(a) Hierarchical
(b) Network
(c) Relational
(d) Object Model

Q3. Which property prevents Alice's balance from being lost during a crash?

(a) Atomicity
(b) Normalisation
(c) Indexing
(d) Access Control

Exit Quiz: 3 Questions — 3 Minutes

Quick check before we wrap up Lecture 02.

Q1. Student data appears in multiple files with the name duplicated. Failure?

(a) Atomicity
(c) Isolation
(b) Redundancy
(d) Security

Q2. Which 1970 model became the foundation of SQL?

(a) Hierarchical
(c) Relational
(b) Network
(d) Object Model

Q3. Which property prevents Alice's balance from being lost during a crash?

(a) Atomicity
(c) Indexing
(b) Normalisation
(d) Access Control

Score 3/3? You are ready for **ER Modeling** on Thursday!

Questions?

"The only bad query is the one left unasked."

Feel free to ask about the file era, RDBMS concepts,
or what to expect in our next SQL labs!

References & Next Steps

Recommended Reading

- **Database System Concepts** (Silberschatz, Korth, Sudarshan)
  Chapter 1: Introduction & Purpose of Database Systems
- **Fundamentals of Database Systems** (Elmasri & Navathe)

Coming Up Next...

⏩ Lecture 03 Preview: ER Modeling

Theoretical Foundations:

- Entities, Attributes, & Relationships
- Cardinality: `1:1`, `1:N`, `M:N`

Hands-on Design:

- Drawing your first ER Diagram
- Domain: Hospital Patient Database